

SPEC-1 - NeuroPathway / Safe Space MVP Database Schema

Child-facing Pattern Recognition AI Reader - PostgreSQL, Safeguarding, AI Review, and Audit Model

Generated: 2026-05-16

Purpose

This PDF captures the concrete MVP database schema for the child-facing Pattern Recognition AI Reader side of NeuroPathway / Safe Space.

The schema is designed for a high-assurance platform for children. It separates child-authored input, AI-derived observations, generated behaviour patterns, child-safe reflections, adult/professional review, safeguarding events, and immutable audit records.

Core architectural principle:

Child input -> AI extraction -> AI pattern -> Child-safe reflection -> Adult review -> Safeguarding escalation -> Audit record

This ensures the system can always answer:

- What did the child actually submit?
- What did the AI infer?
- What evidence created the pattern?
- What did the child see?
- What did the adult/professional see?
- Who reviewed it?
- Was anything escalated?

Database Decision

The application will use PostgreSQL as the primary transactional database.

The initial tenancy model will be a pooled multi-tenant schema, where tenant-owned tables include a mandatory `tenant_id` column. PostgreSQL Row-Level Security will be enabled on sensitive tenant-owned tables to enforce tenant isolation at the database layer.

For early-stage and mid-scale tenants, data will live in shared tables. For enterprise customers, NHS-adjacent partners, local authorities, academy trusts, or high-assurance safeguarding contexts, the architecture must support promotion to a dedicated schema or dedicated database.

Core Database Pattern

- tenants
- users
- tenant_memberships
- roles
- permissions
- audit_logs
- children
- child_checkins

- consent_records
- safeguarding_cases
- safeguarding_events
- ai_outputs
- pattern_observations
- behaviour_patterns
- domain-specific tenant-owned tables with tenant_id
- shared reference tables without tenant_id where appropriate

Isolation strategy:

- Application resolves active tenant from authenticated user/session.
- Database session receives the active tenant context.
- PostgreSQL Row-Level Security policies restrict reads/writes by tenant_id.
- All tenant-owned tables have indexes starting with tenant_id.
- Cross-tenant admin access uses separate privileged service roles with explicit audit logging.
- Child-facing AI workflows must never bypass tenant isolation or safeguarding policy checks.
- AI-derived records must be tenant-scoped and linked back to the source evidence used to generate them.

MVP Schema Overview

The MVP schema supports structured child check-ins, AI safety pre-screening, AI extraction, pattern aggregation, child-safe reflections, adult review, safeguarding escalation, and immutable audit logging.

Logical schemas:

- child_record
- ai
- safeguarding
- audit
- identity
- platform

Concrete SQL Schema

child_record.children

```
create table child_record.children (  
  id uuid primary key,  
  tenant_id uuid not null references platform.tenants(id),  
  display_name text not null,  
  legal_first_name text,  
  legal_last_name text,  
  date_of_birth date,  
  preferred_name text,  
  pronouns text,  
  communication_preferences jsonb not null default '{} '::jsonb,  
  accessibility_preferences jsonb not null default '{} '::jsonb,  
  created_at timestampz not null default now(),  
  updated_at timestampz,  
  archived_at timestampz,  
  sensitivity_tier int not null default 3  
);  
  
create index children_tenant_idx  
on child_record.children (tenant_id);
```

child_record.child_guardians

```
create table child_record.child_guardians (  
  id uuid primary key,  
  tenant_id uuid not null references platform.tenants(id),  
  child_id uuid not null references child_record.children(id),  
  user_id uuid not null references identity.users(id),  
  relationship_type text not null,  
  parental_responsibility boolean not null default false,  
  can_view_child_reflections boolean not null default true,  
  can_view_adult_insights boolean not null default false,  
  can_manage_consent boolean not null default false,  
  created_at timestampz not null default now(),  
  revoked_at timestampz  
);  
  
create unique index child_guardians_unique_active_idx  
on child_record.child_guardians (tenant_id, child_id, user_id)  
where revoked_at is null;
```

child_record.consent_records

```
create table child_record.consent_records (  
  id uuid primary key,  
  tenant_id uuid not null references platform.tenants(id),  
  child_id uuid not null references child_record.children(id),  
  consent_type text not null,  
  lawful_basis text not null,  
  article_9_condition text,  
  granted_by_user_id uuid references identity.users(id),  
  child_assent_status text,  
  applies_to text not null,  
  consent_text_version text not null,  
  granted_at timestampz,  
  withdrawn_at timestampz,  
  expiry_at timestampz,  
  metadata jsonb not null default '{} '::jsonb  
);  
  
create index consent_records_tenant_child_idx  
on child_record.consent_records (tenant_id, child_id, consent_type, granted_at desc);
```

child_record.child_checkins

```
create table child_record.child_checkins (  
  id uuid primary key,  
  tenant_id uuid not null references platform.tenants(id),  
  child_id uuid not null references child_record.children(id),  
  checkin_type text not null,  
  mood_score int check (mood_score between 1 and 5),  
  energy_score int check (energy_score between 1 and 5),  
  sensory_score int check (sensory_score between 1 and 5),  
  stress_score int check (stress_score between 1 and 5),  
  safety_score int check (safety_score between 1 and 5),  
  location_context text,  
  activity_context text,  
  selected_feelings text[] not null default '{}',  
  selected_body_signals text[] not null default '{}',  
  selected_triggers text[] not null default '{}',  
  selected_helpers text[] not null default '{}',
```

```

    free_text text,
    submitted_via text not null default 'child_app',
    visibility text not null default 'child_and_approved_adults',
    created_at timestampz not null default now(),
    sensitivity_tier int not null default 3
);

create index child_checkins_tenant_child_time_idx
on child_record.child_checkins (tenant_id, child_id, created_at desc);

create index child_checkins_tenant_type_time_idx
on child_record.child_checkins (tenant_id, checkin_type, created_at desc);

```

ai.safety_screen_results

```

create table ai.safety_screen_results (
    id uuid primary key,
    tenant_id uuid not null references platform.tenants(id),
    child_id uuid not null references child_record.children(id),
    source_type text not null,
    source_id uuid not null,
    detected_risk_level text not null,
    detected_categories text[] not null default '{}',
    action_taken text not null,
    model_name text not null,
    model_version text not null,
    prompt_version text not null,
    confidence_score numeric,
    created_at timestampz not null default now()
);

create index safety_screen_results_tenant_child_time_idx
on ai.safety_screen_results (tenant_id, child_id, created_at desc);

```

ai.pattern_observations

```

create table ai.pattern_observations (
    id uuid primary key,
    tenant_id uuid not null references platform.tenants(id),
    child_id uuid not null references child_record.children(id),
    source_checkin_id uuid not null references child_record.child_checkins(id),
    observation_type text not null,
    extracted_value jsonb not null,
    confidence_score numeric not null,
    risk_level text not null default 'low',
    model_name text not null,
    model_version text not null,
    prompt_version text not null,
    created_at timestampz not null default now()
);

create index pattern_observations_tenant_child_type_time_idx
on ai.pattern_observations (tenant_id, child_id, observation_type, created_at desc);

```

Example extracted_value JSON

```

{
  "environment": "school_lunch_hall",
  "sensory_factor": "noise",
  "emotion": ["anger", "anxiety"],
  "body_signal": ["shaky"],
  "coping_action": "hid_in_toilet",
  "support_needed": true
}

```

ai.behaviour_patterns

```

create table ai.behaviour_patterns (
    id uuid primary key,
    tenant_id uuid not null references platform.tenants(id),
    child_id uuid not null references child_record.children(id),
    pattern_type text not null,
    pattern_summary text not null,
    child_safe_summary text,
    adult_summary text,
    evidence_window_start timestampz not null,
    evidence_window_end timestampz not null,
    evidence_count int not null check (evidence_count > 0),
    confidence_score numeric not null,
    risk_level text not null,
    status text not null default 'active',
    created_by_job_id uuid,
    created_at timestampz not null default now(),
    reviewed_by_user_id uuid references identity.users(id),
    reviewed_at timestampz
);

```

```
);

create index behaviour_patterns_tenant_child_status_idx
on ai.behaviour_patterns (tenant_id, child_id, status, created_at desc);

create index behaviour_patterns_tenant_risk_idx
on ai.behaviour_patterns (tenant_id, risk_level, created_at desc);
```

ai.pattern_evidence_links

```
create table ai.pattern_evidence_links (
  id uuid primary key,
  tenant_id uuid not null references platform.tenants(id),
  behaviour_pattern_id uuid not null references ai.behaviour_patterns(id),
  source_type text not null,
  source_id uuid not null,
  contribution_reason text,
  created_at timestamptz not null default now()
);

create index pattern_evidence_links_pattern_idx
on ai.pattern_evidence_links (tenant_id, behaviour_pattern_id);
```

ai.ai_outputs

```
create table ai.ai_outputs (
  id uuid primary key,
  tenant_id uuid not null references platform.tenants(id),
  child_id uuid references child_record.children(id),
  ai_job_id uuid not null,
  related_pattern_id uuid references ai.behaviour_patterns(id),
  output_type text not null,
  audience text not null,
  output_text text not null,
  confidence_score numeric,
  risk_level text not null default 'low',
  model_name text not null,
  model_version text not null,
  prompt_version text not null,
  created_at timestamptz not null default now(),
  review_status text not null default 'not_required',
  reviewed_by_user_id uuid references identity.users(id),
  reviewed_at timestamptz,
  review_notes text
);

create index ai_outputs_tenant_child_time_idx
on ai.ai_outputs (tenant_id, child_id, created_at desc);

create index ai_outputs_review_queue_idx
on ai.ai_outputs (tenant_id, review_status, risk_level, created_at desc);
```

ai.child_output_policy_decisions

```
create table ai.child_output_policy_decisions (
  id uuid primary key,
  tenant_id uuid not null references platform.tenants(id),
  child_id uuid not null references child_record.children(id),
  ai_output_id uuid not null references ai.ai_outputs(id),
  decision text not null,
  decision_reason text not null,
  risk_level text not null,
  requires_adult_review boolean not null default false,
  decided_at timestamptz not null default now()
);

create index child_output_policy_decisions_tenant_child_idx
on ai.child_output_policy_decisions (tenant_id, child_id, decided_at desc);
```

safeguarding.adult_review_queue

```
create table safeguarding.adult_review_queue (
  id uuid primary key,
  tenant_id uuid not null references platform.tenants(id),
  child_id uuid not null references child_record.children(id),
  source_type text not null,
  source_id uuid not null,
  review_type text not null,
  priority text not null default 'normal',
  status text not null default 'open',
  assigned_to_user_id uuid references identity.users(id),
  due_at timestamptz,
  created_at timestamptz not null default now(),
  reviewed_at timestamptz,
  reviewed_by_user_id uuid references identity.users(id),
```

```

        outcome text,
        review_notes text
    );

create index adult_review_queue_tenant_status_priority_idx
on safeguarding.adult_review_queue (tenant_id, status, priority, created_at desc);

create index adult_review_queue_child_idx
on safeguarding.adult_review_queue (tenant_id, child_id, created_at desc);

```

safeguarding.safeguarding_events

```

create table safeguarding.safeguarding_events (
    id uuid primary key,
    tenant_id uuid not null references platform.tenants(id),
    child_id uuid not null references child_record.children(id),
    event_type text not null,
    severity text not null,
    source_type text not null,
    source_id uuid,
    summary text not null,
    status text not null default 'open',
    created_by_user_id uuid references identity.users(id),
    created_by_system boolean not null default false,
    created_at timestampz not null default now(),
    assigned_to_user_id uuid references identity.users(id),
    closed_at timestampz,
    closure_reason text
);

create index safeguarding_events_tenant_child_time_idx
on safeguarding.safeguarding_events (tenant_id, child_id, created_at desc);

create index safeguarding_events_tenant_status_severity_idx
on safeguarding.safeguarding_events (tenant_id, status, severity, created_at desc);

```

audit.audit_events

```

create table audit.audit_events (
    id uuid primary key,
    tenant_id uuid not null references platform.tenants(id),
    actor_user_id uuid references identity.users(id),
    actor_role text,
    subject_type text not null,
    subject_id uuid,
    child_id uuid references child_record.children(id),
    action text not null,
    event_time timestampz not null default now(),
    ip_address inet,
    user_agent text,
    lawful_basis text,
    reason text,
    metadata jsonb not null default '{} '::jsonb
);

create index audit_events_tenant_time_idx
on audit.audit_events (tenant_id, event_time desc);

create index audit_events_child_time_idx
on audit.audit_events (tenant_id, child_id, event_time desc);

```

Row-Level Security pattern

```

alter table child_record.child_checkins enable row level security;

create policy tenant_isolation_child_checkins
on child_record.child_checkins
using (
    tenant_id = current_setting('app.current_tenant_id')::uuid
);

-- API layer should set these at the start of each request transaction:
select set_config('app.current_tenant_id', :tenant_id, true);
select set_config('app.current_user_id', :user_id, true);
select set_config('app.current_role', :role, true);

```

Workflow

```

Child submits structured check-in
-> Store child_checkins
-> Safety screen input
-> Store safety_screen_results
-> If high-risk: create safeguarding_event + adult_review_queue item
-> If low/medium-risk: extract pattern_observations
-> Aggregate behaviour_patterns

```

- > Generate ai_outputs
- > Apply child_output_policy_decisions
- > Show child-safe reflection or route to review
- > Write audit_events

Recommended MVP API endpoints

POST /child/checkins
GET /child/reflections
GET /children/{child_id}/patterns
GET /adult/review-queue
POST /adult/reviews/{review_id}/decision
POST /safeguarding/events
GET /audit/children/{child_id}/events